Lab Experiment: Stable Sorting of Patient Scan Data Using Merge Sort

Medical Imaging System — Sorting by Time and Priority

Aim

To analyze how Merge Sort can be applied to sort patient scan data in a medical imaging system based on time and priority, to demonstrate how Merge Sort's stability preserves the correct relative order of patients, and to study the divide-and-conquer approach along with its time complexity to justify its use for this kind of large-scale, accuracy-critical data.

Algorithm :

Merge Sort works on the divide-and-conquer principle, breaking the problem of sorting a large list of patient records into smaller, easier sub-problems, solving each one, and then combining the results back together.

Overall Merge Sort :

1. If the group of patient records being looked at has more than one record, find the middle position of the group and split it into two halves — a left half and a right half.
2. Apply the same sorting process separately to the left half and to the right half. This step repeats itself (recursion), continually splitting each half into smaller halves.
3. Continue splitting until every group contains only a single patient record. A group of one is already considered sorted, since there is nothing to compare it with.
4. Once both halves of a group are individually sorted, merge them back together into one combined, sorted group. This merging step is where the actual comparison and ordering happens.
5. Keep merging sorted halves back together, level by level, until the entire original list of patient records is fully merged into one single sorted list.

The Merge Step :

1. Keep a pointer at the start of the left half and a pointer at the start of the right half.
2. Compare the record currently pointed to in the left half with the record currently pointed to in the right half, using the priority value (lower number means more urgent).
3. Place whichever record has the smaller (more urgent) priority into the combined result, and move that half's pointer forward by one.
4. If both records being compared have the same priority, always place the record from the left half first. Since the left half appeared earlier in the original arrival order, this rule ensures patients who scanned at the same priority level keep their original time-based order — this is what makes the sort stable.
5. Repeat the comparison-and-place step until one half runs out of records.

6. Copy any remaining records from the other half directly into the combined result, since they are already in sorted order.

Why Stability Matters Here

In a medical imaging system, several patients often share the same priority level (for example, several "routine" scans). Stability guarantees that among patients with equal priority, the one who arrived earlier for scanning is still processed earlier after sorting. Without a stable sort, patients with equal priority could be silently reordered, which could unfairly delay someone who arrived first — a serious accuracy and fairness issue in a healthcare setting. Merge Sort is naturally stable as long as the merge step always favors the left half on ties, as shown above.

Time Complexity and Suitability for Large Data

Merge Sort splits the data in half at every level, and merging each level takes time proportional to the number of records at that level. This gives a time complexity of O(n log n) in the best, average, and worst cases, unlike algorithms such as Quick Sort whose worst case can degrade to O(n squared). This consistent, predictable performance is valuable for a medical imaging system, where the volume of scan records can be large and performance must remain reliable regardless of how the data happens to be ordered. The trade-off is that Merge Sort needs extra memory (O(n)) to hold the temporary left and right halves during merging, which is an acceptable cost for the accuracy and predictability it guarantees.

Program (C)

```
#include <stdio.h>
#include <string.h>

#define MAX_NAME 20

typedef struct {
    int patientID;
    char name[MAX_NAME];
```

```
    char scanTime[6];   // format "HH:MM"
    int priority;       // 1 = most urgent, higher number = less urgent
} PatientRecord;

void merge(PatientRecord arr[], int left, int mid, int right) {
    int n1 = mid - left + 1;
    int n2 = right - mid;

    PatientRecord L[n1], R[n2];

    for (int i = 0; i < n1; i++) L[i] = arr[left + i];
    for (int j = 0; j < n2; j++) R[j] = arr[mid + 1 + j];

    int i = 0, j = 0, k = left;

    while (i < n1 && j < n2) {
        /* <= keeps the sort STABLE: if priorities are equal, the record
           from the left half (which was earlier in the original order)
           is placed first. */
        if (L[i].priority <= R[j].priority) {
            arr[k] = L[i];
            i++;
        } else {
            arr[k] = R[j];
            j++;
        }
        k++;
    }

    while (i < n1) { arr[k] = L[i]; i++; k++; }
    while (j < n2) { arr[k] = R[j]; j++; k++; }
}

void mergeSort(PatientRecord arr[], int left, int right) {
    if (left < right) {
        int mid = left + (right - left) / 2;
        mergeSort(arr, left, mid);
        mergeSort(arr, mid + 1, right);
        merge(arr, left, mid, right);
    }
}
```

```
void printRecords(PatientRecord arr[], int n) {
    printf("%-6s %-15s %-10s %-8s\n", "ID", "Name", "ScanTime", "Priority");
    printf("-----------------------------------------\n");
    for (int i = 0; i < n; i++) {
        printf("%-6d %-15s %-10s %-8d\n",
               arr[i].patientID, arr[i].name, arr[i].scanTime, arr[i].priority);
    }
}

int main() {
    PatientRecord patients[] = {
        {101, "Kumar",    "09:15", 2},
        {102, "Priya",    "09:20", 1},
        {103, "Ahmed",    "09:25", 2},
        {104, "Fatima",   "09:30", 3},
        {105, "Ravi",     "09:35", 1},
        {106, "Meena",    "09:40", 2}
    };
    int n = sizeof(patients) / sizeof(patients[0]);

    printf("Input (order in which patients arrived for scanning):\n");
    printRecords(patients, n);

    mergeSort(patients, 0, n - 1);

    printf("\nOutput (sorted by priority, stable on equal priority):\n");
    printRecords(patients, n);

    return 0;
}
```

Sample Input

Patient records in the order they arrived for scanning (ID, Name, Scan Time, Priority — 1 is most urgent):

ID	Name	Scan Time	Priority
101	Kumar	09:15	2
102	Priya	09:20	1
103	Ahmed	09:25	2
104	Fatima	09:30	3
105	Ravi	09:35	1
106	Meena	09:40	2

Output

Patient records sorted by priority. Notice that patients 101, 103, and 106 all share priority 2, and they remain in their original arrival order (09:15, then 09:25, then 09:40) — this is the stability of Merge Sort in action:

ID	Name	Scan Time	Priority
102	Priya	09:20	1
105	Ravi	09:35	1
101	Kumar	09:15	2
103	Ahmed	09:25	2
106	Meena	09:40	2
104	Fatima	09:30	3

Conclusion

Merge Sort proves to be well suited for sorting patient scan data in a medical imaging system. Its divide-and-conquer approach breaks a large scheduling problem into small, manageable comparisons, and its guaranteed O(n log n) time complexity ensures reliable performance even as the volume of scan data grows. Most importantly, its stability ensures that patients with the same priority level are never reordered relative to one another, preserving fairness and accuracy in the order in which they are scanned — a critical requirement in a healthcare setting where trust in the scheduling order matters as much as the speed of sorting.

```c
#include <stdio.h>
#define MAX 100
void merge(int a[], int low, int mid, int high)
{
    int temp[MAX];
    int i = low, j = mid + 1, k = low;
    while (i <= mid && j <= high)
    {
        if (a[i] <= a[j])
            temp[k++] = a[i++];
        else
            temp[k++] = a[j++];
    }
    while (i <= mid)
        temp[k++] = a[i++];
    while (j <= high)
        temp[k++] = a[j++];
    for (i = low; i <= high; i++)
        a[i] = temp[i];
}
void mergeSort(int a[], int low, int high)
{
    if (low < high)
    {
        int mid = (low + high) / 2;
        mergeSort(a, low, mid);
        mergeSort(a, mid + 1, high);
        merge(a, low, mid, high);
    }
}
int main()
{
    int n, i;
    printf("Enter number of patient scan records: ");
    scanf("%d", &n);
    int a[MAX];
    printf("Enter scan priorities/time values:\n");
    for (i = 0; i < n; i++)
        scanf("%d", &a[i]);
    mergeSort(a, 0, n - 1);
    printf("\nSorted Scan Data:\n");
    for (i = 0; i < n; i++)
        printf("%d ", a[i]);
    printf("\n");
    return 0;
}
```

```
7
6 7 5 4 2 1 3
```

Output

```
records: Enter scan
priorities/time values:

Sorted Scan Data:
1 2 3 4 5 6 7
```

Submitted